Comparing Distributed-Memory Programming Frameworks with Radix Sort

Michael Ferguson (HPE), Ryan Friese (PNNL), Shreyas Khandekar (HPE), Matt Drozt (HPE)

November 16th, 2025

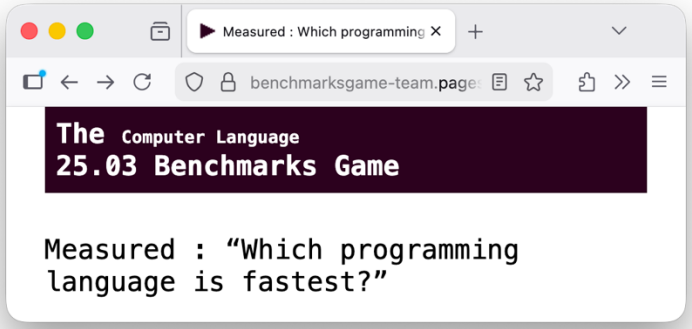
Comparing Distributed-Memory Programming Frameworks

We see a need to compare distributed-memory programming frameworks

- To help programmers choose appropriate tools for their tasks
- To show which problems each framework excels at
- To compare both productivity and performance

Inspiration: the Computer Programming Languages Benchmark Game

- Anybody can contribute improved programs in any of the languages
- Website includes regularly updated performance results



https://benchmarksgame-team.pages.debian.net/benchmarksgame/index.html

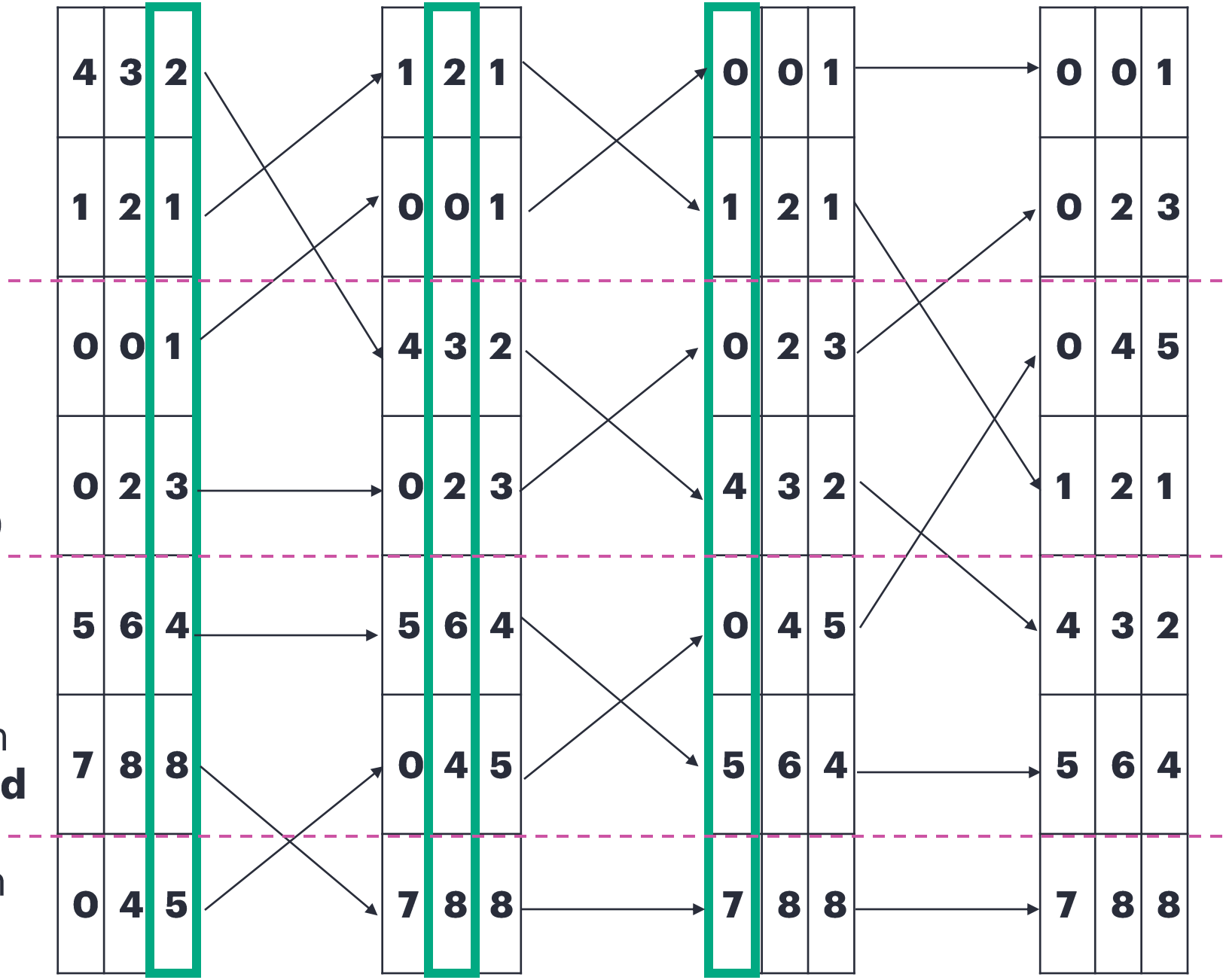
To keep the scope manageable, we focused on a single benchmark

New benchmarks & implementations are welcome in our repository!

- https://github.com/hpc-ai-adv-dev/distributed-lsb/

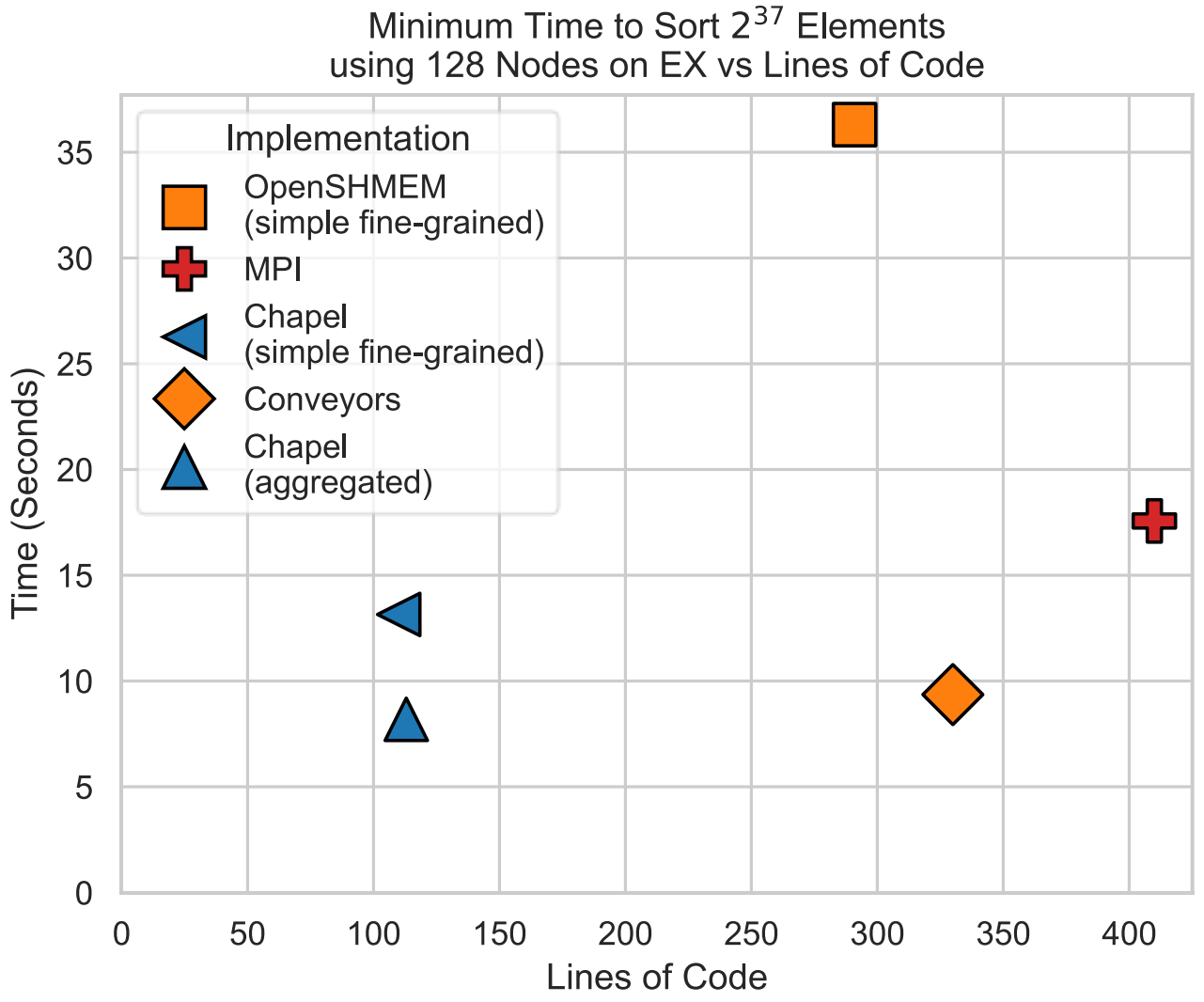
Experiment Overview

- Investigated five distributed programming frameworks on both **HPE Cray EX** (EX) and **InfiniBand** (IB) systems:
  - **Chapel** (with and without aggregation)
  - **OpenSHMEM** (with and without Conveyors for message aggregation)
  - **MPI**
  - **Lamellar** (currently only IB support)
- With each framework, we implemented a communication intensive algorithm: **Distributed Least Significant Digit-First Radix Sort** (LSD Sort) — shown on the right



Methodology

- **Development constraint:** ~1 day of implementation effort per framework by an experienced developer
- **Tuning:** Each LSD radix sort tested under multiple runtime configurations to identify optimal launch parameters
- **Benchmark:** Measured time to sort datasets of **128-bit key/value pairs** across varying data sizes and resource counts



Source Lines of Code and Performance Details

Framework	Version	Lines of Code	EX Sorting Performance[1]	IB Sorting Performance[2]
--- Compared LSD Sorting Implementations ---				
Chapel	Simple, Fine-Grained	★ **110**	10,455	182
Chapel	Aggregated	113	★ **16,782**	★ **2,519**
MPI	(AlltToAll)	410	7,823	1,018
OpenSHMEM	Simple, Fine-Grained	291	3,786	48
Conveyors	Aggregated OpenSHMEM	330	14,687	1,323
Lamellar	Unsafe Array	185	--	475
Lamellar	Atomic Array	175	--	468

1. Sorting 2^{37} elements using 128 Nodes (M elements sorted / second)
2. Sorting 2^{34} elements using 32 Nodes (M elements sorted / second)

The 4-Line Change in the Chapel Version

We showed **multiple variants** with differing line counts and timings

Interestingly, Chapel saw a **1.6-13.8x** improvement with a small four-line change in the original source code

Other frameworks typically required larger code modifications or the use of additional layers to achieve comparable optimization

In contrast, Chapel's standard library offers aggregation purpose-built to allow for incremental refactoring for performance

The **aggregated Chapel implementation** is used as the representative version in this LSD radix sort comparison

...

```
// coforall task begin { ...
    var tD = calcBlock(task, lD.low, lD.high);
    // calc new position and put data there in temp
    {


        for i in tD {
            const ref tempi = temp.localAccess[i];
            const key = comparator.key(tempi);
            var bucket = getDigit(key, rshift, last, negs);
            var pos = taskBucketPos[bucket];
            taskBucketPos[bucket] += 1;
            a[pos] = tempi;
        }


     }
} //coforall task
```

The 4-Line Change in the Chapel Version

We showed **multiple variants** with differing line counts and timings

Interestingly, Chapel saw a **1.6-13.8x** improvement with a small four-line change in the original source code

Other frameworks typically required larger code modifications or the use of additional layers to achieve comparable optimization

In contrast, Chapel's standard library offers aggregation purpose-built to allow for incremental refactoring for performance

The **aggregated Chapel implementation** is used as the representative version in this LSD radix sort comparison

```
+ use CopyAggregation;
...     // coforall task begin { ...
            var tD = calcBlock(task, lD.low, lD.high);
            // calc new position and put data there in temp
            {
+               var aggregator = new DstAggregator(t);
                for i in tD {
                    const ref tempi = temp.localAccess[i];
                    const key = comparator.key(tempi);
                    var bucket = getDigit(key, rshift, last, negs);
                    var pos = taskBucketPos[bucket];
                    taskBucketPos[bucket] += 1;
+/-                 aggregator.copy(a[pos], tempi); // a[pos] = tempi;
                }
+               aggregator.flush();
            }
        } //coforall task
```

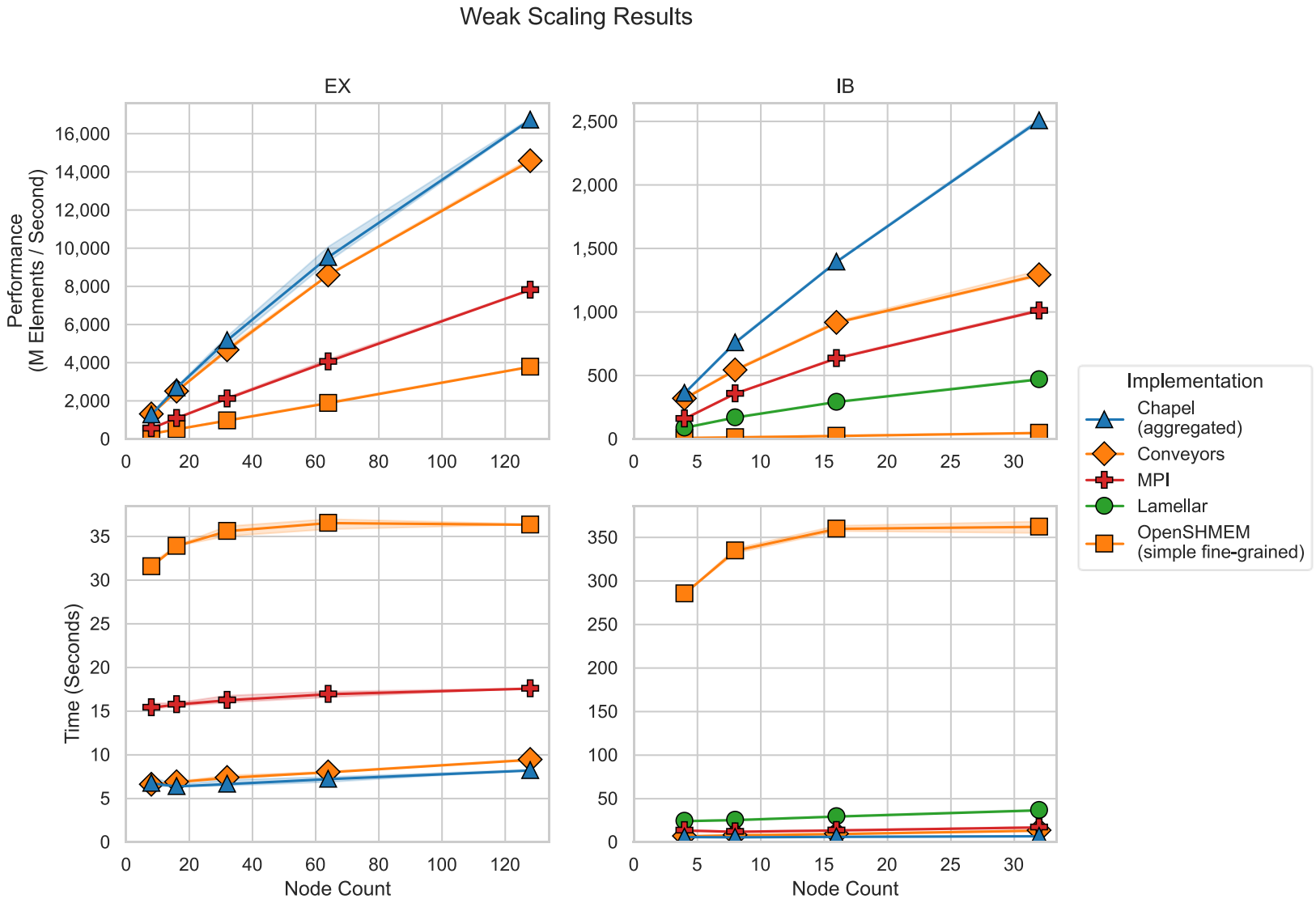
What Changed From OpenSHMEM to Conveyors? One part:

```
for (int64_t i = 0; i < locN; i++) {




  SortElement elt = localPart[i];
  int bucket = getBucket(elt, digit);
  int64_t &next = (*starts)[bucket];
  int64_t dstGlobalIdx = next;
  // store 'elt' into 'dstGlobalIdx'
  auto dst = B.globalIdxToLocalIdx(dstGlobalIdx);
  shmem_putmem(GB + dst.locIdx, &elt, sizeof(SortElement), dst.rank);
  next += 1;
}
```

Adding Conveyors involves control flow adjustment

```
convey_begin(request, sizeof(IdxSortElement), alignof(IdxSortElement));
int64_t i = 0;
while (convey_advance(request, i == locN)) {
  for (; i < locN; i++) {
    SortElement elt = localPart[i];
    int bucket = getBucket(elt, digit);
    int64_t &next = (*starts)[bucket];
    int64_t dstGlobalIdx = next;
    // store 'elt' into 'dstGlobalIdx'
    auto dst = B.globalIdxToLocalIdx(dstGlobalIdx);
    IdxSortElement payload = { .locIdx = dst.locIdx, .value = elt };
    if (! convey_push(request, &payload, dst.rank)) break;
    next += 1;
  }
  IdxSortElement* local;
  while((local = (IdxSortElement*)convey_apull(request, NULL)) != NULL) {
    GB[local->locIdx] = local->value;
  }
}
convey_reset(request);
```

Weak-Scaling Results Across Frameworks



- Figure shows **weak-scaling performance** of the most performant LSD radix sort implementation per framework
  - Sorted 2^{30} elements per node on EX
  - Sorted 2^{29} elements per node on IB
- Under ideal weak scaling, doubling nodes and data size should **increase total performance** while maintaining constant runtime
- All the LSD sort implementations showed good weak scaling
- The aggregated Chapel implementation achieved the **highest performance** on both systems

How Do Other Distributed Sort Implementations Compare?

Framework	Version	Lines of Code	EX Sorting Performance[1]	IB Sorting Performance[2]
--- Compared LSD Sorting Implementations ---				
Chapel	Simple, Fine-Grained	★ **110**	10,455	182
Chapel	Aggregated	113	★ **16,782**	★ **2,519**
MPI	(AlltToAll)	410	7,823	1,018
OpenSHMEM	Simple, Fine-Grained	291	3,786	48
Conveyors	Aggregated OpenSHMEM	330	14,687	1,323
Lamellar	Unsafe Array	185	--	475
Lamellar	Atomic Array	175	--	468

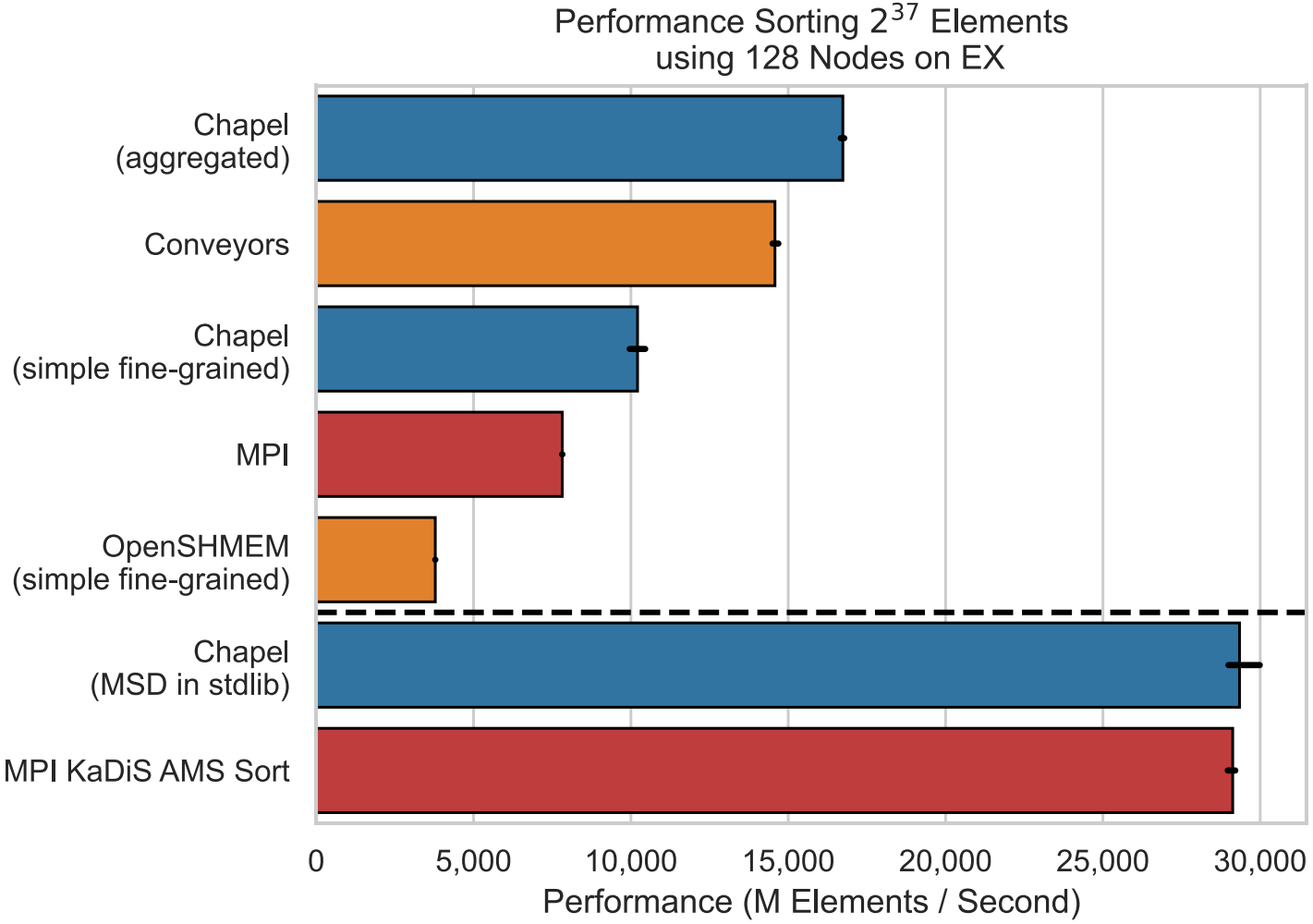
1. Sorting 2^{37} elements using 128 Nodes (M elements sorted / second)
2. Sorting 2^{34} elements using 32 Nodes (M elements sorted / second)

How Do Other Distributed Sort Implementations Compare?

Framework	Version	Lines of Code	EX Sorting Performance[1]	IB Sorting Performance[2]
--- Compared LSD Sorting Implementations ---				
Chapel	Simple, Fine-Grained	★ **110**	10,455	182
Chapel	Aggregated	113	★ **16,782**	★ **2,519**
MPI	(AlltToAll)	410	7,823	1,018
OpenSHMEM	Simple, Fine-Grained	291	3,786	48
Conveyors	Aggregated OpenSHMEM	330	14,687	1,323
Lamellar	Unsafe Array	185	--	475
Lamellar	Atomic Array	175	--	468
--- Additional Sorting Implementations for Comparison ---				
Chapel	MSD Sort (Standard Libary)	2,200	27,555	2,432
MPI	KaDiS AMS Sort	4,200	29,209	--

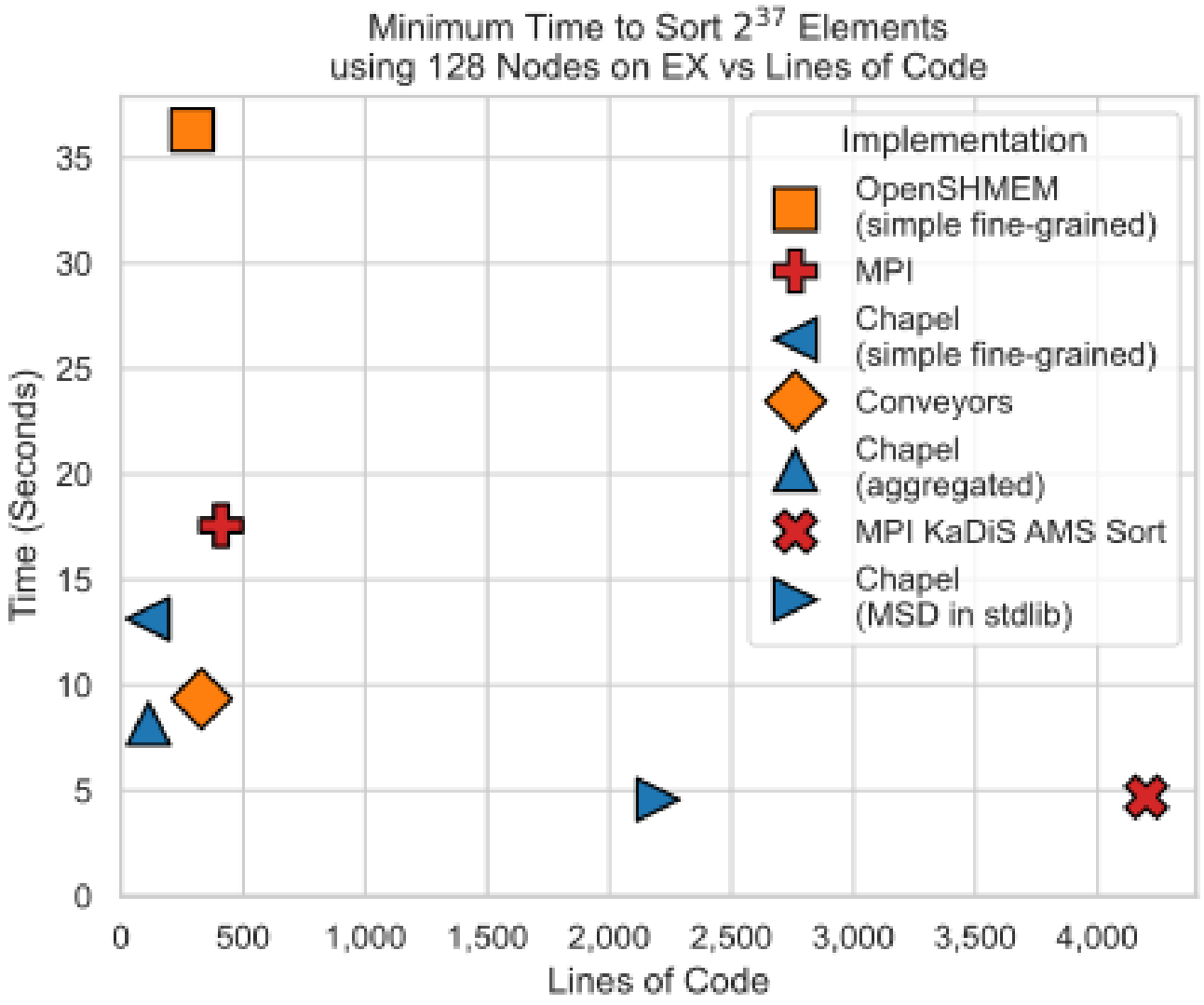
1. Sorting 2^{37} elements using 128 Nodes (M elements sorted / second)
2. Sorting 2^{34} elements using 32 Nodes (M elements sorted / second)

Comparing Performance on 128 EX Nodes



- The bar plot shows the the sort performance when sorting 2^{37} 128-byte elements when using 128 nodes on an EX system
- Bars shown above the dashed line are comparable implementations of an LSD Radix sort
  - Of these, the Chapel version utilizing aggregation was performed the best, approximately **4x** faster than OpenSHMEM and **2x** faster than MPI
  - The Chapel and Conveyors implementations benefit from message aggregation, one-sided communication, and communication overlap
- Bars shown below the dashed line are other, more complex, distributed sorting algorithms for comparison
  - Chapel is capable of being as performant as other more sophisticated distributed sorting algorithms

Comparing Performance and Productivity



- This scatterplot compares
  - time it takes each sorting implementation to sort 2^{37} elements using 128 nodes on an EX system
  - and the number of source lines of code as counted by `cloc`
- Optimal frameworks will approach the origin in the lower left: fewer lines of code indicate increased developer productivity and shorter run times indicating high performance
- Of the LSD sort implementations, the aggregated Chapel version is the most performant while containing ~**4x fewer lines of source code** as compared to MPI
- Shorter codes benefited from native support for distributed arrays and parallel aggregation, aiding developer productivity

A Qualitative Comparison of Examined Frameworks

MPI

- **Pro:** The industry standard, very large developer community
- **Con:** Missing primitives that developers may

Chapel

- **Pro:** Concise ability to create parallel tasks across cores and nodes across a supercomputer
- **Pro:** Documentation and extensive

OpenSHMEM

- **Pro:** Very easy to install
- **Pro:** Fast compile times
- **Con:** Uses a custom *collective* allocator, limiting

Lamellar

- **Pro:** Very easy to install, follows the standard installation path of most Rust crates
- **Pro:** Emphasis on memory safety
- **Pro:** Descriptive (albeit potentially cryptic to newcomers) error messages help to catch bugs at compile time
- **Con:** Currently only supports InfiniBand systems
- **Con:** Long compile times (typical of Rust projects)

Contribute to this Study

- The source code for each of the LSB-Radix sort implementations is open source and available on GitHub
- Repository can be found at: https://github.com/hpc-ai-adv-dev/distributed-lsb/
- Contributions are both welcome and extremely appreciated
- We plan to make this a “living study repository” and hope that as others find or submit optimized versions of LSB-Radix sort we can keep this repository up to date with later findings
- Tell us what we did wrong!! ☺

Thank You – Happy Sorting

Michael Ferguson – michael.ferguson@hpe.com
Ryan Friese – ryan.friese@pnnl.com
Shreyas Khandekar – shreyas.khandekar@hpe.com
Matt Drozt – drozt@hpe.com